



Chain of Responsibility

Design Pattern



About me

- Java developer



What is Chain of Responsibility?

Pass requests through handler chain

Each handler: process OR pass forward

Sender doesn't know specific handler

Loose coupling between components

Request → Handler1 → Handler2 → Result



Monolithic code with many if-else statements

Hard to change the order of checks

Multiple responsibilities

Difficult to test

Problem the Pattern Solves

```
public void processRequest(Request req) {  
    if (condition1) { /* handle */ }  
    else if (condition2) { /* handle */ }  
    else if (condition3) { /* handle */ }  
    // ... many conditions  
}
```



Real-World Example

Tech Support Call:

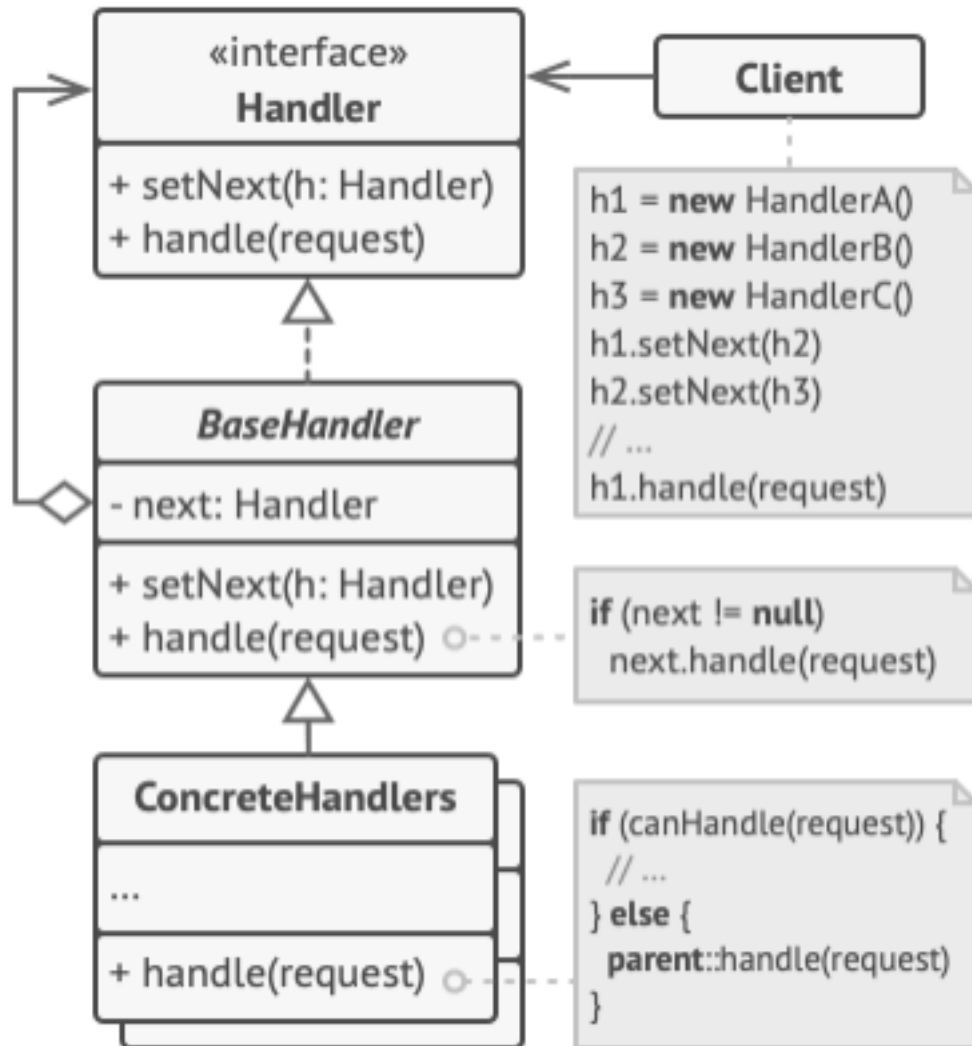
Auto-responder → Level 1 → Level 2 → Engineer

Each level:

- Handles what it can
- Passes complex issues forward
- No level knows the whole chain



Pattern Structure



```
// Handler interface
public interface Handler {
    void handle(Request request);
    void setNext(Handler next);
}

// Concrete Handler
public class ConcreteHandler implements Handler {
    private Handler next;

    public void handle(Request request) {
        if (canHandle(request)) {
            // process
        } else if (next != null) {
            next.handle(request);
        }
    }
}
```

When to Use Chain of Responsibility?

Multiple handlers for request

Processing order matters

Runtime chain changes

Decouple sender/receiver



Pros & Cons

Single Responsibility Principle

Easy to add new handlers

Loose coupling

Runtime flexibility

No processing guarantee

Performance overhead

Complex debugging



CoR vs Other Patterns

Pattern	What's Similar	Key Difference
Command	- Request handling - Decoupling sender/receiver	- CoR: passes through chain of handlers - Command: encapsulates request as object
Decorator	- Chain-like structure - Sequential processing	- CoR: each handler decides to pass or stop - Decorator: ALL decorators always apply
Observer	- Multiple object communication - Loose coupling	- CoR: sequential processing through chain - Observer: parallel notification to ALL
Mediator	- Eliminates direct connections - Centralized control	- CoR: linear chain flow - Mediator: hub-and-spoke communication



Examples

Java Ecosystem:

- Servlet Filters
- Spring Security Chain
- Exception Handling (try-catch)
- Logging Frameworks

Business Example:

ATM Money Dispenser:

\$100 bills → \$50 bills → \$20 bills → \$10 bills

Each handler processes what it can, passes the rest forward



Best Practices

One handler = one responsibility

Add exception handling

Test entire chain

Keep chains reasonably short

Provide fallback handlers

Log processing in each handler



Thank you

- Author: Denys Kuchmei
- My LinkedIn: <https://www.linkedin.com/in/denys-kuchmei-7a8259208/>
- Date: September 2025
- [Join Codeus community in Discord](#)
- [Join Codeus community in LinkedIn](#)

